

Dependency Version Alignment version catalog + platform (BOM) · pitch revamp · target repo is Maven, not Gradle

One file says **what** dependencies exist, one says **which versions** are legal, and modules declare **names only**. So the pitch is not the tool — it is **buy the cheapest level that removes version drift**.

01 · THE GOAL, AND WHO OWNS WHAT

Version catalog	What exists — names, bundles, plugin ids
Platform / BOM	Which versions are legal — alignment + constraints
Modules	Names only — a version here overrides the platform

The catalog is your API. The platform is your policy. Neither is a lockfile.

02 · WHERE THIS REPO ALREADY STANDS

Maven-only: pom.xml + mvnw. Zero Gradle files, so this is a build-system migration, not a config tweak.
It inherits spring-boot-starter-parent 4.0.6, so the Boot BOM is **already imported** and every Boot dependency carries **no version**.
The only two unmanaged deps are pinned once in <properties>: r2dbc-mysql 1.4.2, datafaker 2.4.2.
Net: the stated goal is **already met at single-module scale**.

03 · COST LADDER — BUY ONLY WHAT YOU NEED

Level	Setup	Adopt when
L0	Single module + parent POM (this repo)	Always — free
L1	Multi-module Maven: parent + <modules>	2+ deployables
L2	Gradle: catalog + java-platform + convention plugin + lockfile	2+ apps/teams sharing libs

A four-module Gradle build for one focused demo is structure the problem lacks.

04 · MAVEN EQUIVALENT (ALREADY IN PLACE HERE)

```
<!-- parent POM: the BOM is the "platform", properties are the "catalog" -->
<dependencyManagement><dependencies><dependency>
  <groupId>org.springframework.boot</groupId>
  <artifactId>spring-boot-dependencies</artifactId>
  <version>${spring-boot.version}</version>
  <type>pom</type><scope>import</scope>
</dependency></dependencies></dependencyManagement>
```

Maven has no version catalog; the parent POM plus properties play that role, and this repo already gets it free from spring-boot-starter-parent. The honest pitch is **nothing to migrate** unless you actually need modules.

05 · RECOMMENDATION

Stay on Maven here, and close the real gap instead: add a drift guard (maven-enforcer dependencyConvergence) and versions-maven-plugin for upgrade prompts. Revisit the catalog/platform setup only when the build actually goes multi-module — that is the moment it earns its keep.

06 · REVAMP: WHAT TO FIX IN THE DRAFT

- **No repositories declared** → the build fails; add mavenCentral() in settings.gradle.
- **Plugin version hardcoded in the module** → move it to [plugins] and apply alias(...); also kills the dead [versions] entry.
- **junit = 5.10.0 as a constraint** → the Boot BOM manages higher and Gradle keeps the highest, so the pin silently loses. Use version { strictly ... }.
- **shared uses implementation for a Spring type in its API** → use api, or consumers cannot see it.
- **“BOM ≈ package-lock.json”** → a BOM is constraints; the real lockfile is dependencyLocking + gradle.lockfile.
- **platform(project(':platform')) per module** → a convention plugin in build-logic so it cannot be forgotten.
- **Pins Boot 3.2.0, uses allowDependencies()** → pin the 4.0.6 you run; allowDependencies() is unneeded for constraints-only.

07 · CORRECTED CORE (ONLY WHAT CHANGES)

```
# gradle/libs.versions.toml
[versions]
spring-boot = "4.0.6"
[plugins]
spring-boot = { id = "org.springframework.boot", version.ref = "spring-boot" }
[libraries]
web = { module = "org.springframework.boot:spring-boot-starter-web" }

// settings.gradle -- the draft declared no repositories at all
dependencyResolutionManagement { repositories { mavenCentral() } }

// platform/build.gradle -- constraints only, so no allowDependencies()
plugins { id 'java-platform' }
dependencies { api platform('org.springframework.boot:spring-boot-dependencies:4.0.6') }

// backend/build.gradle -- names only, versions live above
plugins { alias(libs.plugins.spring.boot); id 'java' }
dependencies {
  implementation platform(project(':platform')) // omit and resolution fails
  implementation project(':shared'); implementation libs.web
}
```

08 · WHERE THE UPGRADE ACTUALLY PAYS OFF

- **Convention plugin** in build-logic: every module inherits the platform, catalog and test setup, so the BOM line cannot be forgotten.
- **dependencyLocking** + gradle.lockfile: the reproducibility the draft claims the BOM gives you.
- **CI guard:** ./gradlew dependencies --write-locks, then fail the build on drift.

Verified against the target repo, not assumed: it is Maven-only (pom.xml + mvnw, no Gradle files anywhere), it inherits spring-boot-starter-parent 4.0.6, and its only pinned third-party versions are r2dbc-mysql 1.4.2 and datafaker 2.4.2.